



फ्रैंकपास: क्लाइट-साइड स्टेटलेस आर्किटेक्चर एवं मास्टर मणिकान्त यादव का क्रिप्टोग्राफिक स्थिरांक (*MMY* constant)



मास्टर मणिकान्त यादव (Master Manikant Yadav)

भूमिका: क्रिप्टोग्राफिक सिस्टम आर्किटेक्ट व संस्थापक · mastermanikant.com/research
शोध एवं रचना: जनवरी 2026 · प्रकाशन: 15 सितम्बर 2026 · विनिर्देश v1.2: 16 सितम्बर 2026
ईमेल: connect@mastermanikant.com · GitHub: github.com/Mastermanikant/frank-pass

सामग्री तालिका (Table of Contents)

सार (Abstract)	पृष्ठभूमि एवं समस्या का विवरण
स्टेटलेस आर्किटेक्चर एवं डिटरमिनिज्म	MMY_constant की औपचारिक परिभाषा
क्रिप्टोग्राफिक व्युत्पत्ति पाइपलाइन (6-चरण)	गोल्डन बेस-32 पूल एवं शून्य मॉड्यूलो बायस
पासवर्ड एन्ट्रॉपी सूत्र एवं सुरक्षा विश्लेषण	उपयोगिता सुरक्षा, MMY नॉर्मलाइजेशन एवं सीक्रेट की खोने का समाधान
थ्रेट मॉडल एवं एंडपॉइंट सुरक्षा मैट्रिक्स	आधिकारिक क्रिप्टोग्राफिक टेस्ट वेक्टर्स
आजीवन गवर्नेंस एवं फ्रोजन कोर नियम	निष्कर्ष एवं स्वतंत्र सत्यापन प्रकटीकरण
आधिकारिक मानक एवं सन्दर्भ (References)	

📖 तकनीकी शब्द संक्षिप्त शब्दकोश (Quick Glossary)

KDF (Key Derivation Function)	की-व्युत्पत्ति फलन - एक गणितीय प्रक्रिया जो पासफ्रेज़ से मजबूत पासवर्ड बनाती है।
PBKDF2	10,00,000 इटरेशन्स पर कॉन्फ़िगर की-डेरिवेशन फलन (KDF)।
HMAC-SHA512	512-बिट आउटपुट वाला क्रिप्टोग्राफिक MAC एवं PRF (स्पूडौरैंडम फंक्शन) निर्माण।
Stateless (स्टेटलेस)	ऐसी प्रणाली जो कुछ भी सेव नहीं करती, हर बार गणना से पासवर्ड बनाती है।
Domain Salt (डोमेन साल्ट)	लक्षित प्लेटफॉर्म पहचानकर्ता जो प्रत्येक सेवा के लिए स्वतंत्र क्रैंडैशियल सुनिश्चित करता है।
Application Domain Salt (एप्लिकेशन साल्ट)	सार्वजनिक एप्लिकेशन-स्तरीय डोमेन सेपरेटर जो क्रॉस-एप्लिकेशन टकराव को रोकता है।
Normalization (सामान्यीकरण)	इनपुट की स्वचालित सफाई - केवल a-z, 0-9 रखना।
Entropy (एन्ट्रॉपी)	अनिश्चितता का माप - जितने बिट्स अधिक, उतना अनुमान लगाना कठिन।

सार (Abstract)

पारंपरिक क्लाउड पासवर्ड मैनेजर्स की सबसे बड़ी कमजोरी है: **Single Point of Failure (विफलता का अकेला केंद्र)**। वे आपके सभी पासवर्ड्स को एन्क्रिप्ट करके किसी रिमोट सर्वर पर सेव करते हैं। जब भी सर्वर हैक होता है, हमलावर पूरा डेटाबेस डाउनलोड कर लेता है और अपने कंप्यूटरों पर ऑफलाइन क्रैकिंग शुरू कर देता है। इसके अलावा, क्लाउड रिकवरी के नाम पर मिलने वाले SMS OTP और ईमेल रीसेट लिंक सोशल इंजीनियरिंग के नए खतरे पैदा करते हैं।

फ्रैंकपास पासवर्ड स्टोर करने के इस पूरे झंझट और खतरे को आर्किटेक्चरल रूप से समाप्त करता है। यह एक **100% क्लाइट-साइड, ओपन और स्टेटलेस पासवर्ड जनरेशन सिस्टम** है, जो सर्वर या डिवाइस पर कभी भी पासवर्ड को सेव नहीं करता।

फ्रैंकपास **कैनोनिकल प्रोटोकॉल पैरामीटर टुपल (Canonical Protocol Parameter Tuple)** को परिभाषित करता है, जिसे औपचारिक रूप से **मास्टर मणिकान्त यादव का क्रिप्टोग्राफिक स्थिरांक ($\mathcal{M}\mathcal{Y}_{\text{constant}}$)** नामित किया गया है: यह 7 अनिवार्य पैरामीटर्स का एक स्थायी और फ्रोजन आर्किटेक्चरल टुपल है जो PBKDF2 के 10,00,000 (10 लाख) राउंड्स, HMAC-SHA512 स्ट्रीम विस्तार, इनपुट नॉर्मलाइजेशन (स्पेस, केस और सिंबल्स की स्वचालित सफाई), यूनिकोड NFC स्टैंडर्ड और शून्य-मॉड्यूलो-बायस गोल्डन बेस-32 कैरेक्टर पूल ($2^5 = 32$) को जोड़ता है।

यह श्वेतपत्र (Whitepaper) फ्रैंकपास के गणितीय सिद्धांतों, 80 से 100 बिट्स एन्ट्रॉपी सूत्रों, थ्रेट मॉडल और आजीवन डिटरमिनिज्म नियमों को प्रस्तुत करता है, जिससे यूजर बिना इंटरनेट और बिना 1 बाइट डेटा भेजे दशकों तक वही सटीक पासवर्ड दोबारा निकाल सकते हैं।

1. पृष्ठभूमि एवं समस्या का विवरण (Background & Problem Statement)

पारंपरिक पासवर्ड मैनेजर एन्क्रिप्टेड सर्वर वॉल्ट्स पर काम करते हैं: $C_i = E_K(P_i)$, जहाँ एन्क्रिप्टेड डेटा C_i को क्लाउड इंफ्रास्ट्रक्चर पर स्टोर किया जाता है। यह मॉडल 3 बड़ी बुनियादी कमजोरियों से ग्रस्त है:

- सर्वर डेटा चोरी का खतरा:** क्लाउड सर्वर पर लाखों यूज़र्स के वॉल्ट्स का जमा होना हैकर्स के लिए बड़ा निशाना (Honeypot) बन जाता है। सर्वर ब्रीच होते ही हमलावर डेटाबेस डाउनलोड कर लेता है और अपने सुपरकंप्यूटर पर जितने दिन चाहे ऑफलाइन क्रैकिंग चला सकता है।
- रिकवरी बैकडोर्स:** सर्वर पर मौजूद 'Forgot Password', SMS OTP या ईमेल रीसेट लिंक मास्टर पासवर्ड को बायपास कर देते हैं, जिससे सिम-स्वैप और सोशल इंजीनियरिंग के जरिए अकाउंट्स आसानी से हैक हो जाते हैं।
- कंपनी पर निर्भरता:** यदि पासवर्ड मैनेजर कंपनी बंद हो जाए, सर्वर डाउन हो या वे अचानक फीस बढ़ा दें, तो यूजर अपने ही पासवर्ड्स का एक्सेस खोने के जोखिम में आ जाता है।

फ्रैंकपास पासवर्ड स्टोरेज को **शुद्ध क्लाइट-साइड गणितीय गणना (Key Derivation)** से बदलकर इन सभी कमियों को जड़ से समाप्त करता है।

2. स्टेटलेस आर्किटेक्चर एवं डिटरमिनिज्म स्थिरांक (Determinism Invariant)

स्टेटलेस का सीधा अर्थ है: **शून्य स्टोरेज (Zero Storage)**। पासवर्ड को कभी भी किसी डेटाबेस, ब्राउज़र कुकी, लोकल फाइल या क्लाउड में सेव नहीं किया जाता। इसके बजाय, यूजर के इनपुट्स और एक स्थायी क्रिप्टोग्राफिक टुपल की मदद से उसी समय ऑन-डिमांड पासवर्ड की गणना की जाती है:

$$\text{Password} = f(\text{MasterSecret}, \text{Platform}, \text{Username}, \text{Version}, \mathcal{MMY}_{\text{constant}})$$

अपरिवर्तनीयता की अनिवार्यता ($\mathcal{MMY}_{\text{constant}}$ को क्यों लॉक किया गया है)

ईमानदार तकनीकी स्पष्टीकरण: मास्टर मणिकान्त यादव का स्थिरांक ($\mathcal{MMY}_{\text{constant}}$) कोई प्राकृतिक भौतिक नियम नहीं है, बल्कि यह मास्टर मणिकान्त यादव द्वारा स्थापित एक **औपचारिक क्रिप्टोग्राफिक स्थिरांक (Architectural Invariant)** है।

स्टेटलेस सिस्टम में यदि कभी भी कोई आंतरिक पैरामीटर (जैसे PBKDF2 राउंड्स, हैश फंक्शन या कैरेक्टर टेबल) बदल जाए, तो गणितीय आउटपुट पूरी तरह बदल जाएगा और यूजर को इंटरनेट पर अपने सभी अकाउंट्स के पासवर्ड रीसेट करने पड़ेंगे। इस टुपल को हमेशा के लिए लॉक करना यह गारंटी देता है कि आज बना पासवर्ड 10, 20 या 50 साल बाद भी किसी भी कंप्यूटर पर बिल्कुल सटीक और वही दोबारा बनेगा।

3. कैनोनिकल प्रोटोकॉल पैरामीटर टुपल (FrankPass v1 विनिर्देश / $\mathcal{MMY}_{\text{constant}}$) की औपचारिक परिभाषा

मान लीजिए $\mathcal{U}$ यूजर की मास्टर सीक्रेट की है, $\mathcal{D}$ लक्षित वेबसाइट या प्लेटफॉर्म है, $\mathcal{U}$ यूजरनेम या ईमेल है, $\mathcal{V} \in \mathbb{N}^+$ वर्जन काउंटर है, और $\mathcal{P} \in \mathbb{N}^+$ पासवर्ड की लंबाई है।

फ्रैंकपास का **कैनोनिकल प्रोटोकॉल पैरामीटर टुपल (Canonical Protocol Parameter Tuple)**, जिसे औपचारिक रूप से **मास्टर मणिकान्त यादव का क्रिप्टोग्राफिक स्थिरांक ($\mathcal{MMY}_{\text{constant}}$)** नामित किया गया है, 7 अनिवार्य फ़ोर्जन तत्वों का टुपल है:

$$\mathcal{MMY}_{\text{constant}} = \langle \text{APP_ID}, \text{VER}, \mathcal{I}_{10^6}, \mathcal{H}_{\text{SHA512}}, \mathcal{C}_{\text{entropy}}, \mathcal{N}_{\text{norm}}, \mathcal{P}_{\text{NFC}} \rangle$$

टुपल घटक	औपचारिक मान	क्रिप्टोग्राफिक व सिस्टम उद्देश्य
APP_ID	"MasterManikant_PassGen"	एप्लिकेशन डोमेन सेपरेटर, जो क्रॉस-एप्लिकेशन रेनबो टेबल हमलों को रोकता है।
VER	"v1"	कोर प्रोटोकॉल वर्जन टैग।
$\mathcal{I}_{10^6}$	10,00,000 राउंड्स	PBKDF2 कार्य कारक (Work Factor), जो GPU ब्रूट-फोर्स खोज पर भारी कम्प्यूटेशनल लागत लगाता है।
$\mathcal{H}_{\text{SHA512}}$	HMAC-SHA512	512-बिट स्क्रूडरैडम फंक्शन (PRF), जिसका उपयोग की-डेरिवेशन और स्ट्रीम विस्तार के लिए होता है।
$\mathcal{C}_{\text{entropy}}$	Golden Base-32 Alphabet	भ्रम-मुक्त 32-कैरेक्टर पूल ($2^5 = 32$), जो मॉड्यूलो बायस और टाइपिंग गलतियों को समाप्त करता है।
$\mathcal{N}_{\text{norm}}$	MMY Normalization	इनपुट नॉर्मलाइजेशन (लोअरकेस, स्पेस-सफाई, सिंबल-सफाई) जो टाइपिंग की मानवीय गलतियों को बेअसर करता है।
$\mathcal{P}_{\text{NFC}}$	Unicode Standard NFC	यूनिकोड नॉर्मलाइजेशन, जो विभिन्न ऑपरेटिंग सिस्टमों पर मल्टी-बाइट विसंगतियों को रोकता है।

* **बेंचमार्क कार्यप्रणाली:** Chromium 128 WebCrypto SubtleCrypto पर सामान्य तापमान में 100 लगातार परीक्षणों का औसत। GPU श्रृंखला Hashcat v6.2.6 (मोड 12000: PBKDF2-HMAC-SHA512, 10 लाख राउंड्स) को NVIDIA RTX 4090 संदर्भ हार्डवेयर पर मापकर निकाला गया।

4. क्रिप्टोग्राफिक व्युत्पत्ति पाइपलाइन (6-चरण प्रवाह)



फ्रैंकपास का क्रिप्टोग्राफिक इंजन W3C WebCrypto API (`crypto.subtle`) के माध्यम से पूरी तरह आपके ब्राउज़र में निष्पादित होता है। इसकी 6-चरणीय प्रक्रिया नीचे दी गई है:

FRANKPASS CRYPTOGRAPHIC DERIVATION PIPELINE

```

1. Secret Key Normalization (frankpass-core.js):
S_norm = S.toLowerCase().replace(/\s+/g, '').replace(/[^\a-z0-9]/g, '') |
U_norm = U.toLowerCase().replace(/\s+/g, '')
D_norm = D.toLowerCase().replace(/\s+/g, '')

2. Local Domain Key Derivation (getLocalPepper):
FRANKPASS_DOMAIN_SALT = "FrankbaseSuperSecretMango2026!" (PUBLIC)
[यह गोपनीय नहीं है - GitHub पर सार्वजनिक है - Kerckhoffs' Principle] |
dataStr = "Version=v1|User=" + U_norm + "|Plat=" + D_norm + "|Key=" + S_norm
hmac = HMAC_SHA512( FRANKPASS_DOMAIN_SALT, dataStr )
preKey = hex( hmac )
FOR i = 0 TO 999:
    preKey = hex( SHA256( preKey + FRANKPASS_DOMAIN_SALT ) )
appDomainKey = preKey (64-character hexadecimal string)

3. Context Vector Construction (Length-prefixed Netstring encoding):
CTX = "22:MasterManikant_PassGen|2:v1|len(D):D_norm|len(U):U_norm|
    len(appDomainKey):appDomainKey|len(V):V|len(profile):profile|len(L):L" |
seedData = UTF8_Encode( CTX.normalize('NFC') )

4. Primary Key Derivation (PBKDF2-HMAC-SHA512, 1M Iterations):
baseKey = ImportKey( raw, UTF8_Encode( appDomainKey.normalize('NFC') ) ) |
PRK = PBKDF2( baseKey, Salt=seedData, Iterations=1,000,000, SHA-512 )

5. Continuous HMAC Stream Expansion (HKDF-style expansion):
prkKey = ImportKey( raw, PRK, HMAC-SHA512 )
T_1 = HMAC_SHA512( prkKey, 0x01 )
T_2 = HMAC_SHA512( prkKey, T_1 || 0x02 )
BYTE_STREAM = T_1 || T_2 (128 pseudorandom bytes total)

6. Modulo Bias-Free Base-32 Character Mapping:
validMax = 256 - (256 % 32) = 256 (गणितीय रूप से 0.000% हानि)
For each byte b: char = charset[b % 32]
Enforce mandatory set presence (Upper, Lower, Number, Symbol)
OUTPUT_PASSWORD = Validated Mapped Characters (length L)

```

💡 FRANKPASS_DOMAIN_SALT (सार्वजनिक एप्लिकेशन डोमेन विभाजक) : विनिर्देश

उपरोक्त पाइपलाइन में `FRANKPASS_DOMAIN_SALT` का मान है:

```
FRANKPASS_DOMAIN_SALT = "FrankbaseSuperSecretMango2026!"
```

- यह एक सार्वजनिक (Public) स्थिरांक है (GitHub रिपॉजिटरी में खुले में उपलब्ध)।
- यह गुप्त नहीं है और न ही इसे गुप्त रखने की आवश्यकता है: सुरक्षा का आधार यूजर की मास्टर सीक्रेट की है, यह स्थिरांक नहीं।
- इसका उद्देश्य है क्रॉस-एप्लिकेशन डोमेन पृथक्करण (Cross-App Domain Separation) - किसी अन्य KDF एप्लिकेशन के साथ टकराव रोकना।
- यह Kerckhoffs का सिद्धांत (1883) पालन करता है: "एल्गोरिथ्म और स्थिरांक सार्वजनिक हों, सुरक्षा केवल की की मजबूती पर निर्भर हो।"

5. गोल्डन बेस-32 पूल एवं शून्य मॉड्यूलो बायस का गणितीय प्रमाण

पारंपरिक रैंडम पासवर्ड जनरेटर जो 256 बाइट्स को किसी मनमाने कैरेक्टर पूल (जैसे 55, 62 या 94 अक्षरों) पर मैप करते हैं, वे **मॉड्यूलो बायस (Modulo Bias)** की बीमारी से ग्रस्त होते हैं, जिससे कुछ अक्षरों के चुने जाने की संभावना दूसरों से अधिक हो जाती है।

फ्रैंकपास एक सटीक **गोल्डन बेस-32 कैरेक्टर पूल** ($\mathcal{C}$, जहाँ $|\mathcal{C}| = 32 = 2^5$) का उपयोग करके इस समस्या को गणितीय रूप से हल करता है:

कैटेगरी	पूल में शामिल कैरेक्टर (Used)	संख्या	हटाए गए भ्रामक कैरेक्टर (Excluded)	हटाने का वैज्ञानिक कारण (भ्रामक जोड़े / Confusion Pairs)
बड़े अक्षर (Uppercase)	A D E F H L M N R T	10	B, C, G, I, J, K, O, P, Q, S, U, V, W, X, Y, Z	B-8, O-0-Q, I-1-1, S-5, C-c, K-k, P-p, X-x, Z-z, U-V-W, G-6, J-1, Y-v जैसे दृश्य भ्रमों को समाप्त करने हेतु
छोटे अक्षर (Lowercase)	d e f h m t	6	a, b, c, g, i, j, k, l, n, o, p, q, r, s, u, v, w, x, y, z	l-1-I, o-0-O, s-5-S, b-d-p-q (दर्पण/समान आकार), c-o, u-v-w, n-m-h, a-o के टाइपिंग भ्रम से बचाव
अंक (Digits)	2 3 4 6 7 8 9	7	0, 1, 5	0 (O-0-o), 1 (I-I-1), 5 (S-s-5) के भ्रम को 100% रोकने हेतु
विशेष प्रतीक (Symbols)	@ # \$ % & * + = ?	9	' " ` () { } [] < > / \ ~ ^ _ ; : , . - _	' " ` (कोड्स बग्स), ((ब्रेकेट पेयर्स), - (डैश-अंडरस्कोर), / \ (स्लैश), : ; , . (विराम चिह्न) से बचाव
कुल गोल्डन बेस-32 कैरेक्टर पूल ($ \mathcal{C} $)		32	सटीक $2^5 = 32$ (शून्य मॉड्यूलो बायस व प्रति कैरेक्टर 5-बिट एन्ट्रॉपी)	

$$|\mathcal{C}| = 10 + 6 + 7 + 9 = 32 = 2^5 \text{ कैरेक्टर}$$

शून्य मॉड्यूलो बायस एवं कैरेक्टर-पॉलिसी मैपिंग का गणितीय विश्लेषण:

मान लीजिए $B \in \{0, 1, \dots, 255\}$ HMAC-SHA512 द्वारा उत्पन्न एक समान वितरित स्यूडोरैंडम बाइट है। अधिकतम संरक्षित सीमा `validMax` इस प्रकार ज्ञात होती है:

$$\text{validMax} = 256 - (256 \bmod 32) = 256 - 0 = 256$$

चूँकि 256 संख्या 32 का पूर्ण पूर्णांक गुणज है ($2^8 / 2^5 = 2^3 = 8$ सटीक पूर्ण चक्र):

$$\forall c \in \mathcal{C}, \quad P(\text{byteToChar}(B) = c) = 8 / 256 = 1 / 32 = 0.03125 \text{ (3.125\%)}$$

प्राथमिक मैपिंग में प्रत्येक कैरेक्टर के चुने जाने की संभावना बिल्कुल $1/32$ (3.125%) होती है, जिससे मॉड्यूलो बायस हानि **0.000%** है।

अनिवार्य कैरेक्टर-सेट नीति प्रवर्तन (Mandatory Policy Enforcement):

मानक प्रोफाइल ('standard') में 4 कैरेक्टर श्रेणियों (बड़ा अक्षर, छोटा अक्षर, अंक, प्रतीक) की अनिवार्य उपस्थिति सुनिश्चित करने के लिए `frankpass-core.js` एक डिटरमिनिस्टिक 2-चरणीय सत्यापन लागू करता है:

1. **प्राथमिक स्ट्रीम मैपिंग:** 128-बाइट स्यूडोरैंडम स्ट्रीम के पहले L बाइट्स को बेस-32 वर्णों में मैप किया जाता है।

2. डिटरमिनिस्टिक इंडेक्शन: यदि L वर्णों में कोई अनिवार्य श्रेणी अनुपस्थित हो, तो स्ट्रीम के अंतिम बाइट्स (`revIdx`) का उपयोग करके एक अद्वितीय डिटरमिनिस्टिक इंडेक्स पर अनुपस्थित श्रेणी का वर्ण प्रतिस्थापित कर दिया जाता है।

16-कैरेक्टर बेस-32 का सर्च स्पेस 80 बिट्स ($32^{16} = 2^{80} \approx 1.21 \times 10^{24}$) है। अनिवार्य श्रेणी प्रवर्तन वैध सेट को सीमित करता है; प्रभावी सुरक्षा मास्टर सीक्रेट एन्ट्रापी और सर्च स्पेस की न्यूनतम सीमा पर निर्भर करती है: **Effective Security** = $\min(H_{\text{secret}}, H_{\text{search}})$ ।

6. पासवर्ड एन्ट्रापी सूत्र एवं सुरक्षा विश्लेषण

चूँकि गोल्डन बेस-32 पूल का प्रत्येक कैरेक्टर ठीक 5 बिट्स की सूचना एन्ट्रापी रखता है ($\log_2(32) = 5.000$ bits), इसलिए पासवर्ड की सर्च स्पेस जटिलता लंबाई के साथ रैखिक रूप से बढ़ती है:

$$H_{\text{search}} = L \times \log_2(32) = 5 \times L \text{ bits}$$

पासवर्ड लंबाई (L)	सर्च स्पेस कॉम्बिनेशन	सर्च एन्ट्रापी (H_{search})	क्रैकिंग समय @ 10^9 अनुमान/सेकंड	समय @ 10^6 PBKDF2/सेकंड (क्लस्टर)
12 कैरेक्टर	$32^{12} = 2^{60} \approx 1.15 \times 10^{18}$	60.000 bits	~36.5 वर्ष	~36,500 वर्ष
16 कैरेक्टर (मानक)	$32^{16} = 2^{80} \approx 1.21 \times 10^{24}$	80.000 bits	~38.3 मिलियन वर्ष (3.83×10^7 वर्ष)	~ 3.83×10^{10} वर्ष
20 कैरेक्टर (विस्तारित)	$32^{20} = 2^{100} \approx 1.27 \times 10^{30}$	100.000 bits	~40.2 ट्रिलियन वर्ष (ब्रह्मांड की आयु से अधिक)	~ 4.02×10^{16} वर्ष

डिटरमिनिस्टिक संदर्भ पृथक्करण:

फ्रैंकपास में ऑफलाइन प्री-इमेज सर्च स्पेस यूजर व डोमेन वैरिएबल्स के नेटस्ट्रिंग्स का उपयोग करता है:

Context Tuple = (APP_ID, VER, D_{norm}, U_{norm}, appDomainKey, V, Profile, L)

डोमेन, यूजरनेम और वर्जन वैरिएबल्स डिटरमिनिस्टिक संदर्भ पृथक्करण प्रदान करते हैं जो क्रॉस-सर्विस टकराव व रेनबो टेबल को रोकते हैं। लेकिन जब वे किसी विशिष्ट खाते को लक्षित करने वाले हमलावर को ज्ञात हों, तो आधारभूत खोज कठिनाई यूजर की मास्टर सीक्रेट की एन्ट्रापी द्वारा शासित होती है।

7 से 9 शब्दों का पासफ्रेज़ मॉडल (40 से 60+ प्रभावी एन्ट्रॉपी बिट्स)

जटिल प्रतीकों (जैसे P@\$\$w0rd!#) की बनावटी सुरक्षा के भ्रम से बाहर निकलें। प्राकृतिक व्यक्तिगत शब्द संयोजन बिना टाइपिंग बाधा के उच्च सुरक्षा प्रदान करते हैं:

विचार करें एक 9-शब्दों के प्राकृतिक वाक्य पर: "mere door ke mosa ka beta ram bahut achha aadmi hai"

- व्यावहारिक पासफ्रेज़ एन्ट्रॉपी:** प्राकृतिक भाषाई संरचना के कारण मानव-निर्मित वाक्य में लगभग 40 से 60+ बिट्स की प्रभावी एन्ट्रॉपी होती है। 100+ बिट्स की पूर्ण क्रिप्टोग्राफिक एन्ट्रॉपी हेतु यूजर्स बहु-शब्दीय यादृच्छिक डाइसवेयर (Diceware) पासफ्रेज़ का उपयोग कर सकते हैं।
- शून्य टाइपिंग गलतियाँ:** बिना किसी अजीब सिंबल के साधारण शब्द होने के कारण मोबाइल पर टाइप करना अत्यंत सहज है और MMY Normalization स्पेस व केस की चिंता समाप्त कर देता है।
- फ्रैंकपास प्रो लोकल वॉल्ट के साथ तालमेल:** यूजर को यह वाक्य केवल एक बार याद रखना होता है। फ्रैंकपास प्रो में यह पासफ्रेज़ आपके डिवाइस पर ही लोकल एन्क्रिप्टेड वॉल्ट (AES-256-GCM) में सुरक्षित रहता है और केवल 4-अंकों के पिन या बायोमेट्रिक्स से 1 सेकंड में अनलॉक होकर ऑटो-फिल हो जाता है।

महत्वपूर्ण अंतर: सीक्रेट की एन्ट्रॉपी बनाम आउटपुट पासवर्ड

अनुप्रयुक्त क्रिप्टोग्राफी का मूलभूत नियम है कि की-डेरिवेशन फ़ंक्शन (KDF) शून्य में से एन्ट्रॉपी नहीं बना सकता। वास्तविक सुरक्षा का नियम है:

$$\text{Effective Security} = \min(H_{\text{passphrase}}, H_{\text{search}})$$

16 अक्षरों का आउटपुट (80-बिट सर्व स्पेस) टकराव को रोकता है, लेकिन यदि किसी वेबसाइट से पासवर्ड चोरी हो जाए तो हमलावर आपके इनपुट पासफ्रेज़ पर ब्रूट-फोर्स हमला करेगा। 10 लाख राउंड्स का PBKDF2 अटैकर को भारी धीमा करता है, लेकिन वास्तविक सुरक्षा आपकी सीक्रेट की की मजबूती पर टिकी होती है।

अनुशंसा: यूजर्स को 5 से 6 यादृच्छिक शब्दों (डाइसवेयर / EFF स्टाइल, ~65 से 80 बिट्स वास्तविक एन्ट्रॉपी) या 7 से 9 शब्दों के व्यक्तिगत स्मरणीय वाक्यों का चयन करना चाहिए।

KDF चयन का वैज्ञानिक कारण, हार्डवेयर बेंचमार्क एवं कम्प्यूटेशनल लागत (Empirical Benchmarks):

की-डेरिवेशन फ़ंक्शन के रूप में 10,00,000 (10 लाख) राउंड्स के साथ PBKDF2-HMAC-SHA512 का चयन क्लाइंट-साइड ब्राउज़र परिवेश, शून्य-निर्भरता (Zero-Dependency) और हार्डवेयर-त्वरित WebCrypto API (crypto.subtle) के सटीक संतुलन पर आधारित है:

परीक्षण हार्डवेयर एवं वातावरण	प्लेटफ़ॉर्म / चिपसेट	10 लाख राउंड्स निष्पादन समय (Latency)	यूज़र अनुभव (UX Impact)
हाई-एंड वर्कस्टेशन / पीसी	Apple M2 / Intel Core i7-13700K	~32 ms - 45 ms	तात्कालिक (Sub-50ms 1-क्लिक ऑटो-फिल)
मिड-रेंज लैपटॉप / डेस्कटॉप	AMD Ryzen 5 5600 / Intel i5-1135G7	~55 ms - 75 ms	अत्यंत सहज (Zero UI Lag)
फ्लैगशिप स्मार्टफोन (Mobile)	Snapdragon 8 Gen 2 / Apple A16 Bionic	~78 ms - 95 ms	सहज मोबाइल अनुभव (< 0.1 सेकंड)
बजट स्मार्टफोन (Entry Android)	MediaTek Helio G99 / Snapdragon 680	~140 ms - 210 ms	व्यावहारिक रूप से स्वीकार्य (< 0.25 सेकंड)

हमलावर के विरुद्ध ब्रूट-फोर्स कम्प्यूटेशनल लागत विश्लेषण (Adversary Work Factor):

10 लाख HMAC-SHA512 राउंड्स हमलावर के ऑफलाइन क्रैकिंग पर भारी कम्प्यूटेशनल और वित्तीय अवरोध (Economic Barrier) लगाते हैं:

- **आधुनिक GPU थ्रूपुट (NVIDIA RTX 4090 Hashcat Benchmark):** 10 लाख PBKDF2-HMAC-SHA512 राउंड्स पर एक RTX 4090 GPU कार्ड की अधिकतम गति केवल **~12,500 हैश प्रति सेकंड** तक सीमित हो जाती है।
- **1 अरब (10⁹) गेस प्रयासों की लागत:** एक एकल RTX 4090 को 1 अरब संभावित पासफ्रेज चेक करने में **~22.2 GPU-घंटे** का समय और भारी बिजली/क्लाउड लागत लगेगी।
- **80-बिट पासफ्रेज सुरक्षा:** 6 शब्दों के सामान्य पासफ्रेज (~80 बिट्स एन्ट्रॉपी) को क्रैक करने के लिए 1.20×10^{24} गणनाओं की आवश्यकता होगी, जिसे दुनिया के सबसे बड़े सुपरकंप्यूटर क्लस्टर भी अरबों वर्षों में क्रैक नहीं कर सकते।

💡 इस बेंचमार्क का सटीक दायरा एवं संदर्भ (Scope & Threat Context):

1. यह समय किस चीज़ का है? यह गणना किसी वेबसाइट के 16-अक्षर वाले आउटपुट पासवर्ड की नहीं है, बल्कि यदि किसी थर्ड-पार्टी साइट से पासवर्ड लीक हो जाए तो उससे **यूज़र की मूल मास्टर सीक्रेट की (Master Secret Key) का अनुमान लगाने / रिवर्स-क्रैक करने (Known-Plaintext Pre-image Attack)** की कम्प्यूटेशनल लागत है।

2. ऑनलाइन बनाम ऑफलाइन सुरक्षा का अंतर: लाइव वेबसाइटों पर हमलावर 3 से 5 गलत प्रयासों के बाद रेट-लिमिट (Rate Limiting) व अकाउंट लॉकआउट से स्वतः ब्लॉक हो जाता है। ऊपर दिया गया GPU बेंचमार्क उस सबसे खराब स्थिति (Worst-Case Offline Breach) का विश्लेषण है जहाँ हमलावर लीक हुए डेटाबेस को डाउनलोड करके अपने निजी सुपरकंप्यूटर पर बिना किसी लॉकआउट पाबंदी के ऑफलाइन क्रैकिंग चला रहा हो।

PBKDF2 बनाम Argon2id / Scrypt का आर्किटेक्चरल विश्लेषण:

हालाँकि Argon2id मेमोरी-हार्डनेस (ASIC प्रतिरोध) प्रदान करता है, लेकिन ब्राउज़र परिवेश में Argon2id को चलाने के लिए भारी WebAssembly (WASM) बाइनरी लोड करने और क्लाइंट डिवाइस पर उच्च RAM आवंटित करने की आवश्यकता होती है, जो सीमित संसाधनों वाले मोबाइल ब्राउज़रों में मेमोरी थ्रॉटलिंग का कारण बन सकती है। इसके विपरीत, **PBKDF2-HMAC-SHA512** ब्राउज़र के मानक W3C WebCrypto API (नेटिव C++ इंजन) में शून्य बाहरी निर्भरता के साथ चलता है, जिससे सभी डेस्कटॉप और मोबाइल डिवाइसेस पर 100% सुसंगत और विश्वसनीय प्रदर्शन प्राप्त होता है।

7. उपयोगिता सुरक्षा एवं MMY नॉर्मलाइजेशन का विज्ञान

स्टेटलेस पासवर्ड सिस्टम्स में मानवीय टाइपिंग की भिन्नता सबसे बड़ी समस्या होती है। मोबाइल कीबोर्ड अक्सर:

- पहला अक्षर अपने-आप बड़ा कर देते हैं (जैसे **MySecret** बनाम **mysecret**)।
- स्मार्ट कोट्स या अजीब विराम चिह्न डाल देते हैं।
- ऑटो-कम्प्लीट सुझावों में अंत में स्पेस जोड़ देते हैं।

नॉर्मलाइजेशन के बिना एक भी स्पेस पूरे पासवर्ड को बदल देता है। फ्रैंकपास गणना से पहले **MMY व्यापक इनपुट नॉर्मलाइजेशन** ($\mathcal{N}_{\text{norm}}$) और **यूनिफाइड NFC कंपोज़िशन** ($\mathcal{P}_{\text{NFC}}$) लागू करता है:

```
// Exact Canonical Normalization Implementation:
const normalizedSecret = (secretKey || '')
  .toLowerCase()
  .replace(/\s+/g, '')
  .replace(/[^a-z0-9]/g, '');
```

यह सुनिश्चित करता है कि **"Master Secret 2026! "**, **"mastersecret2026"**, और **"master secret 2026"** हमेशा एक ही बाइट एरे में बदलें, जिससे टाइपिंग की गलती शून्य हो जाती है।

नॉर्मलाइजेशन का क्रिप्टोग्राफिक ट्रेड-ऑफ

ईमानदार तकनीकी प्रकटीकरण: नॉर्मलाइजेशन मास्टर सीक्रेट के कैरेक्टर सेट को केवल 36 अल्फान्यूमेरिक अक्षरों ([a-z0-9]) तक सीमित करता है। सिंबल्स या स्पेस की जटिलता नॉर्मलाइजेशन के दौरान हट जाती है।

इसलिए यूजर्स को अजीब सिंबल्स (जैसे P@\$\$w0rd!) पर निर्भर रहने के बजाय **पासफ्रेज़ की लंबाई और शब्दों की संख्या** बढ़ानी चाहिए (जैसे mango river copper guitar 2026 अत्यंत सुरक्षित है)।

🔑 सीक्रेट की भूल जाने का डर - पूर्ण पारदर्शी समाधान (4 सुरक्षा कवच)

फ्रैंकपास में "सीक्रेट की खोने" की समस्या को आर्किटेक्चरल स्तर पर 4 परतों में हल किया गया है। प्रत्येक परत स्वतंत्र रूप से काम करती है:

कवच 1: MMY नॉर्मलाइजेशन (स्वचालित त्रुटि-शून्य इनपुट)

फ्रैंकपास के नॉर्मलाइजेशन इंजन की सटीक सार्वजनिक कार्यविधि:

- केवल a-z और 0-9 स्वीकार किए जाते हैं - सभी सिंबल्स (@, !, # आदि) स्वतः हट जाते हैं।
- सभी बड़े अक्षर (Capital Letters) स्वतः छोटे हो जाते हैं - `MySecret` = `mysecret` ।
- सभी स्पेस (खाली जगह) स्वतः हट जाते हैं - `"mere ghar ke log"` = `"meregharke.log"` ।

"Mere Door Ke MOSA Ka Beta!" = "meredoorkemosakabeta"

"mere door ke mosa ka beta " = "meredoorkemosakabeta"

"MERE DOOR KE MOSA KA BETA" = "meredoorkemosakabeta"

→ तीनों से बिल्कुल एक ही पासवर्ड बनेगा

व्यावहारिक अर्थ: यूजर कभी भी "गलत" सीक्रेट की टाइप नहीं कर सकता जब तक कि शब्द पूरी तरह अलग न हों। केस, स्पेस, सिंबल्स - सब माफ।

कवच 2: स्मरणीय प्राकृतिक भाषा मॉडल (Secret Key Guide)

फ्रैंकपास की सीक्रेट की गाइड यूजर्स को सिखाती है कि सीक्रेट की को जटिल बनाने के बजाय **प्राकृतिक और स्मरणीय** बनाएं:

- **50+ वास्तविक श्रेणी-वार उदाहरण** - बचपन, परिवार, दैनिक जीवन, संकल्प - जो कोई भूल नहीं सकता।
- **7 से 9 शब्दों का पारिवारिक संबंध वाक्य** एक बार बोला तो ताउम्र याद रहेगा (उदा. "मेरे दूर के मोसा का बेटा राम")।
- **"असंभव कल्पनाशील वाक्य" रणनीति** - जो केवल आपकी कल्पना में है वह न कोई डिक्शनरी जानता है न AI।

कवच 3: कैनोनिकल टेस्ट वेक्टर्स (स्व-सत्यापन प्रणाली)

यदि कोई यूजर अपनी सीक्रेट की के बारे में अनिश्चित हो, तो वे **अनुभाग 9 के आधिकारिक टेस्ट वेक्टर्स** का उपयोग करके तुरंत जांच सकते हैं:

- वेक्टर #1 में `mastersecret2026` + `google.com` से हमेशा `F6h?%Ld48&&e4f?f` बनना चाहिए।
- यदि यह आउटपुट मेल खाता है, तो आपकी सीक्रेट की और सेटअप सही है।
- यह सत्यापन 100% ऑफलाइन है - कोई डेटा बाहर नहीं जाता।

कवच 4: FrankPass Pro लोकल एन्क्रिप्टेड वॉल्ट (4-PIN सुविधा)

उन यूजर्स के लिए जो कभी भी टाइप करने की झंझट नहीं चाहते:

- सीक्रेट की को डिवाइस पर AES-GCM एन्क्रिप्शन से लोकल वॉल्ट में स्टोर करें।
- केवल 4-अंकों का PIN या बायोमेट्रिक (फिंगरप्रिंट) से 1 सेकंड में 1-क्लिक पासवर्ड।
- सीक्रेट की सर्वर पर कभी नहीं जाती - सब कुछ डिवाइस पर ही रहता है।

सारांश: उच्च-एन्ट्रॉपी डिटरमिनिस्टिक सुरक्षा + 1 सेकंड की सुविधा का सर्वोत्तम संगम।

8. थ्रेट मॉडल एवं एंडपॉइंट सुरक्षा मैट्रिक्स (Threat Model)

🔒 औपचारिक क्रिप्टोग्राफिक सुरक्षा मान्यताएं (Formal Cryptographic Assumptions)

- **मान्यता A1 (PRF मजबूती):** HMAC-SHA512 एक सुरक्षित स्पूडोरैंडम फंक्शन (PRF) है जिसके विरुद्ध कोई व्यावहारिक शॉर्टकट हमला ज्ञात नहीं है।
- **मान्यता A2 (हमलावर का दायरा):** हमलावर ऑफलाइन है और केवल किसी लीक हुए डेटाबेस से प्राप्त आउटपुट पासवर्ड से मास्टर की रिवर्स करने की कोशिश कर रहा है (Known-Plaintext Pre-image Attack)।
- **मान्यता A3 (रनटाइम अखंडता व टाइमिंग सुरक्षा):** प्रमुख ब्राउज़र इंजन W3C WebCrypto API प्रिमिटिव्स को नेटिव C++ (जैसे BoringSSL, NSS) में निष्पादित करते हैं; फ्रैंकपास मानक कॉन्स्टेंट-टाइम निष्पादन और टाइमिंग-अटैक प्रतिरोध के लिए होस्ट क्रिप्टोग्राफिक लाइब्रेरी पर निर्भर करता है।

1. सर्वर ब्रीच से पूर्ण मुक्ति (Zero-Storage Immunity):

चूँकि फ्रैंकपास के सर्वर पर 0 पासवर्ड्स, 0 कीज़, और 0 फाइल्स (0 बाइट स्टोरेज) होती हैं, इसलिए सर्वर हैक होने पर भी हमलावर को शून्य (0) डेटा मिलता है। डेटाबेस चोरी का खतरा आर्किटेक्चरल रूप से विलोपित (Architecturally Eliminated) हो जाता है, क्योंकि सर्वर पर क्रेडेंशियल डेटा का अस्तित्व ही नहीं होता।

2. डोमेन पृथक्करण एवं सीमाएं (Domain Separation & Threat Boundary):

डोमेन नाम को लेंथ-प्रिफिक्स नेटस्ट्रिंग में साल्ट करने से अलग-अलग प्लेटफॉर्म ($D_1 \neq D_2$) के पासवर्ड्स गणितीय रूप से असंबंधित होते हैं। यह क्रॉस-साइट पासवर्ड पुनःउपयोग (Password Reuse) और रेनबो-टेबल हमलों को प्रभावी रूप से निष्प्रभावी करता है। यदि वेबसाइट A का डेटाबेस लीक हो जाए, तो हमलावर उससे वेबसाइट B के पासवर्ड का अनुमान नहीं लगा सकता।

क्रिप्टोग्राफिक सीमा: डोमेन पृथक्करण मास्टर सीक्रेट के क्रैक होने से सुरक्षा नहीं देता। यदि हमलावर किसी एक पासवर्ड से मास्टर सीक्रेट की को ब्रूट-फोर्स करने में सफल हो जाता है, तो वह उसी सीक्रेट से सभी अन्य डोमेन्स के पासवर्ड्स निकाल सकता है। इसलिए सुरक्षा की वास्तविक नींव मास्टर सीक्रेट की एन्ट्रॉपी पर आधारित है।

3. शून्य-नेटवर्क एवं एयर-गैप्ड आर्किटेक्चर (Zero-Storage & Zero Network Calls by Design):

फ्रैंकपास वेब ऐप (PWA), डेस्कटॉप ऐप, मोबाइल ऐप और एक्सटेंशन को आर्किटेक्चरल रूप से **शून्य नेटवर्क कॉल्स (Zero Outbound Credential Requests)** के साथ डिज़ाइन किया गया है। सर्वर पर पासवर्ड या क्रेडेंशियल भेजने का कोई एंडपॉइंट न होने के कारण, सर्वर-साइड डेटाबेस चोरी का खतरा पूरी तरह समाप्त हो जाता है।

सर्विस वर्कर द्वारा कैश होने या स्टैंडअलोन ऐप के रूप में इंस्टॉल होने के बाद, फ्रैंकपास बिना इंटरनेट (Airplane Mode) में भी दशकों तक 100% डिटरमिनिस्टिक रूप से वही सटीक पासवर्ड्स व्युत्पन्न करता है।

4. एंडपॉइंट थ्रेट मैट्रिक्स एवं अवशिष्ट सीमाएं:

खतरा (Threat Vector)	फ्रैंकपास का निवारण उपाय	अवशिष्ट सीमाएं एवं सुरक्षा दिशानिर्देश (Guidelines)
क्लिपबोर्ड स्नूपिंग	एक्सटेंशन डायरेक्ट DOM ऑटो-फिल / एंड्रॉइड ऑटो-फिल	ऑपरेटिंग सिस्टम क्लिपबोर्ड लिसनर्स को बायपास करता है, लेकिन <code><all_urls></code> परमिशन वाले दुर्भावनापूर्ण एक्सटेंशन से सुरक्षा नहीं दे सकता। फ्रैंकपास का समाधान: बैंकिंग, पेमेंट्स व प्राथमिक ईमेल जैसे संवेदनशील कार्यों के लिए एक अलग 'क्लीन ब्राउज़र प्रोफाइल' (जिसमें 0 थर्ड-पार्टी एक्सटेंशन हों और केवल फ्रैंकपास हो) का उपयोग करें या फ्रैंकपास स्टैंडअलोन नेटिव ऐप (Windows/Android) का उपयोग करें जो ब्राउज़र सैंडबॉक्स से बाहर स्वतंत्र रूप से चलती है।
थर्ड-पार्टी एक्सटेंशन जासूसी (DOM Snooping)	क्लीन ब्राउज़र प्रोफाइल एवं स्टैंडअलोन नेटिव ऐप्स (Windows & Android)	सत्यनिष्ठ समाधान: ब्राउज़र सैंडबॉक्स में एक एक्सटेंशन दूसरे एक्सटेंशन को ब्लॉक नहीं कर सकता। इसलिए बैंकिंग, पेमेंट्स और प्राथमिक ईमेल (Gmail) जैसे संवेदनशील कार्यों के लिए यूज़र्स को एक 'Clean Dedicated Browser Profile' (जिसमें 0 फालतू एक्सटेंशन हों और केवल फ्रैंकपास हो) या फ्रैंकपास नेटिव ऐप का उपयोग करना चाहिए जो ब्राउज़र सैंडबॉक्स से बाहर पूरी तरह सुरक्षित चलती है।
मेमोरी स्कैपिंग	क्षणिक रैम गणना व बेस्ट-एफर्ट जीरोइज़ेशन	जावास्क्रिप्ट का गारबेज कलेक्टर मेमोरी स्वीप से पहले रैम से तुरंत बाइट्स मिटाने की 100% भौतिक गारंटी नहीं देता।
फ़िशिंग व क्लोन वेबसाइट्स	सख्त डोमेन बाइंडिंग (Strict Domain Binding)	फ्रैंकपास 100% ऑफलाइन है और इसे यह नहीं पता कि साइट असली है या नकली। लेकिन पासवर्ड की गणना सीधे डोमेन नाम से होने के कारण, नकली डोमेन (जैसे <code>g00gle.com</code>) पर अपने-आप पूरी तरह अलग डमी पासवर्ड बनता है, जिससे असली पासवर्ड सुरक्षित रहता है।
हार्डवेयर कीलॉगर्स	वर्चुअल ऑन-स्क्रीन कीबोर्ड व एक्सटेंशन ऑटो-फिल	फिजिकल कीबोर्ड स्पाईवेयर से बचाता है, लेकिन ओएस-स्तरीय कर्नल मैलवेयर से सुरक्षा लोकल डिवाइस की जिम्मेदारी है।

5. उच्च-सुरक्षा खातों (Banking, Gmail, Crypto) हेतु टियर-आधारित सुरक्षा मानक:

फ्रैंकपास यूज़र्स को अपने खातों को दो स्तरों (Tiers) में बांटने की स्पष्ट सलाह देता है:

- टियर 1 (दैनिक सामान्य वेबसाइट्स - Forums, Shopping, Entertainment):** सामान्य ब्राउज़र में फ्रैंकपास एक्सटेंशन या वेब ऐप का उपयोग करें।
- टियर 2 (उच्च-जोखिम खाते - NetBanking, Primary Gmail, Crypto Wallets):** इनके लिए ब्राउज़र में एक अलग 'Clean Dedicated Profile' बनाएं (बिना किसी थर्ड-पार्टी PDF कनवर्टर, एड-ब्लॉकर या कूपन एक्सटेंशन के) अथवा **फ्रैंकपास स्टैंडअलोन डेस्कटॉप/एंड्रॉइड ऐप** का उपयोग करें जो ऑपरेटिंग सिस्टम के सुरक्षित सैंडबॉक्स (Android `FLAG_SECURE`) में चलती है।

हार्डवेयर-आधारित लाइसेंसिंग दर्शन: यूज़र्स को बिना किसी अतिरिक्त खर्च के स्वच्छ सुरक्षा प्रोफ़ाइल (Clean Profile Defense) अपनाने के लिए प्रेरित करने हेतु फ्रैंकपास प्रो का 1 PC लाइसेंस पूरे कंप्यूटर हार्डवेयर (Device ID) से बाउंड होता है। उसी एक पीसी पर यूज़र जितने चाहे ब्राउज़र (Chrome, Edge, Brave), अलग-अलग क्लीन प्रोफाइल्स और आगामी Windows Desktop App को उसी 1 लाइसेंस की से बिना किसी अतिरिक्त शुल्क के चला सकते हैं।

6. "स्टेटलेस विरोधाभास" एवं मेटाडेटा प्रबंधन:

स्टेटलेस प्रणाली में यूजर को डोमेन, यूजरनेम और वर्जन काउंटर (V) का ध्यान रखना पड़ता है। जब त्रैमासिक पासवर्ड बदलना हो (V=1 से V=2), तो एक साधारण ऑफलाइन गैर-संवेदनशील इंडेक्स फाइल (जैसे `github.com: v2` या फ्रैंकपास प्रो लोकल प्रोफाइल) रखना एक व्यावहारिक आवश्यकता है। यह फ्रैंकपास को "स्टेटलेस क्रेडेंशियल पेलोड + वैकल्पिक लोकल मेटाडेटा" के रूप में स्थापित करता है।

9. आधिकारिक क्रिप्टोग्राफिक एवं एंड-टू-एंड टेस्ट वेक्टर्स (Official Test Vectors)

फ्रैंकपास की पूर्ण एंड-टू-एंड पाइपलाइन (डोमेन व इनपुट नॉर्मलाइजेशन + `frankpass-core.js` 10 लाख PBKDF2 राउंड्स + गोल्डन बेस-32 मैपिंग) के 5 मुख्य कैनोनिकल टेस्ट वेक्टर्स नीचे तालिका में दिए गए हैं (पूर्ण स्वचालित टेस्ट सूट में सभी 18 कैनोनिकल वेक्टर्स शामिल हैं):

1. वेक्टर #1: बाइट-दर-बाइट इंटरमीडिएट स्टेट सत्यापन (Byte-for-Byte Canonical Trace)

रॉ इनपुट्स: Key = `"mastersecret2026"`, Platform = `"google.com"`, User = `"user@example.com"`, V = 1, Profile = `"standard"`, L = 16

1. MMY सामान्यीकृत स्लग: $S_{\text{norm}} = \text{"mastersecret2026"}$, $D_{\text{norm}} = \text{"google"}$, $U_{\text{norm}} = \text{"user@example.com"}$

2. लोकल डोमेन की (appDomainKey, 64 Hex Chars):

`3a1ea6d5b148ccc141048ebfa19d61c970fef40b180d53199849e2b9357e7525`

3. कॉन्टेक्स्ट नेटस्ट्रिंग (CTX):

`"22:MasterManikant_PassGen|2:v1|6:google|16:user@example.com|64:3a1ea6d5b148ccc141048ebfa19d61c970fef40b180d53199849e2b9357e7525|1:1|8:standard|2:16"`

4. PBKDF2-HMAC-SHA512: 10,00,000 Iterations (Key = appDomainKey NFC, Salt = CTX NFC, 64 Derived Bytes)

5. कैनोनिकल आउटपुट पासवर्ड: `F6h?%Ld48&&e4f?f`

टेस्ट वेक्टर	मास्टर सीक्रेट (रॉ इनपुट)	रॉ प्लेटफॉर्म इनपुट	सामान्यीकृत स्लग (Normalized Slug)	यूजरनेम	वर्जन	कैनोनिकल आउटपुट पासवर्ड
वेक्टर #1 (Google SSO)	<code>mastersecret2026</code>	<code>google.com / mail.google.com</code>	<code>google</code>	<code>user@example.com</code>	1	<code>F6h?%Ld48&&e4f?f</code>
वेक्टर #2 (GitHub)	<code>Correct Horse Battery Staple 2026!</code>	<code>github.com / https://github.com/login</code>	<code>github</code>	<code>developer</code>	1	<code>tTM38@@6e7f*EMN?</code>
वेक्टर #3 (रोटेशन v2)	<code>Correct Horse Battery Staple 2026!</code>	<code>github.com</code>	<code>github</code>	<code>developer</code>	2	<code>f#RAmLf4?7hhL#26</code>
वेक्टर #4 (Amazon Retail)	<code>secret2026</code>	<code>amazon.com / amazon.in</code>	<code>amazon</code>	<code>user</code>	1	<code>M6@Tfm8D&\$@?A#EL</code>
वेक्टर #5 (AWS Cloud Isolation)	<code>secret2026</code>	<code>aws.amazon.com</code>	<code>aws</code>	<code>user</code>	1	<code>&F9Me@=N+Df37tet</code>

10. आजीवन गवर्नेंस एवं फ्रोजन कोर नियम

यूजर्स को आजीवन पासवर्ड स्थिरता का आश्वासन देने के लिए मास्टर मणिकान्त यादव ने **फ्रैंकपास कोर इनवेरिेंट नियम** स्थापित किया है:

- `frankpass-core.js` में परिभाषित और `MMU_constant` द्वारा शासित क्रिप्टोग्राफिक पाइपलाइन स्थायी रूप से फ्रोजन है।
- भविष्य का कोई भी सॉफ्टवेयर अपडेट या लाइब्रेरी अपग्रेड इस गणितीय व्युत्पत्ति तर्क को नहीं बदलेगा।
- भविष्य के सभी नए फीचर्स (जैसे UI थीम्स, एक्सटेंशन, ऑटोफिल टूल्स) इस फ्रोजन कोर के बाहरी उपभोक्ता के रूप में कार्य करेंगे।

11. निष्कर्ष एवं स्वतंत्र सत्यापन प्रकटीकरण

फ्रैंकपास यह प्रमाणित करता है कि व्यक्तिगत पासवर्ड सुरक्षा के लिए किसी भी तरह के क्लाउड वॉल्ट्स की आवश्यकता नहीं है। **मास्टर मणिकान्त यादव के स्थिरांक** (`MMU_constant`), क्लाइंट-साइड WebCrypto और गोल्डन बेस-32 मैपिंग द्वारा फ्रैंकपास क्लाउड वॉल्ट्स के खतरों को समाप्त कर डेटाबेस स्टोरेज को शुद्ध गणितीय पुनरुत्पादन से बदल देता है।

ईमानदार प्रकटीकरण एवं स्वतंत्र ऑडिट: फ्रैंकपास का पूरा सोर्स कोड MIT लाइसेंस के तहत GitHub पर सार्वजनिक है। इसका कोई भी घटक बंद या गुप्त नहीं है। दुनिया का कोई भी क्रिप्टोग्राफर, सुरक्षा शोधकर्ता या छात्र अनुभाग 9 के आधिकारिक कैनोनिकल टेस्ट वेक्टर्स के साथ इसे स्वतंत्र रूप से गणितीय रूप से सत्यापित कर सकता है।

आधिकारिक संदर्भ कार्यान्वयन:

ओपन-सोर्स कोड: <https://github.com/Mastermanikant/frank-pass>

संस्थापक रिसर्च हब: <https://mastermanikant.com/research/>

12. आधिकारिक मानक एवं सन्दर्भ (Standards & References)

- RFC 8018 / PKCS #5 v2.1:** Moriarty, K., et al. "PKCS #5: Password-Based Cryptography Specification Version 2.1" (PBKDF2-HMAC-SHA512). IETF, 2017.
- NIST SP 800-132:** Meltz, D. "Recommendation for Password-Based Key Derivation". National Institute of Standards and Technology (NIST), 2010.
- RFC 2104:** Krawczyk, H., Bellare, M., & Canetti, R. "HMAC: Keyed-Hashing for Message Authentication". IETF, 1997.
- Unicode Standard TR#15:** Davis, M., & Whistler, K. "Unicode Normalization Forms (NFC Specification)". Unicode Consortium.
- W3C Web Cryptography API:** Watson, M. "Web Cryptography API - W3C Recommendation" (SubtleCrypto Interface), 2017.
- Kerckhoffs का सिद्धांत (1883):** Auguste Kerckhoffs, "La Cryptographie Militaire" (खुला आर्किटेक्चर, गोपनीय केवल मास्टर की)।

🔒 क्रिप्टोग्राफिक प्रायर-आर्ट एवं प्रोटोकॉल अखंडता फिंगरप्रिंट

कैनोनिकल प्रोटोकॉल पहचानकर्ता: `FRANKPASS-SPEC-V1-20260915`

आविष्कारक व प्रायर-आर्ट धारक: मास्टर मणिकान्त यादव (Master Manikant Yadav)

डिजिटल ऑब्जेक्ट मूल: फ्रैंकबेस अनुप्रयुक्त क्रिप्टोग्राफी आर्काइव (mastermanikant.com/research)
ओपन-सोर्स आर्काइव: github.com/Mastermanikant/frank-pass